pqtoolkit

Identifier, Hash, and Encryption Primitives
for the Post-Quantum Era

Umbrella whitepaper covering PQID, PQHash, and PQCrypt

Bahram Samani

pqtoolkit monorepo maintainer

Umbrella Whitepaper, Version 1.0

Released 2026-05-21

Build date: May 21, 2026 **Build time:** 18:21 UTC

Source: `latex/main.tex` **Spec version:** v1.0

Project artifacts: PQID (v1.1, variants `0x01`/`0x02`, 41 B); PQHash (v1.0, variant `0x03`, 73 B); PQCrypt (v1.3, variant `0x04`, suites `0x01`/`0x02`/`0x06`).

This document is the cross-project overview. Each project has its own normative whitepaper: `pqid/latex/PQID_Whitepaper.pdf`, `pqhash/latex/PQHash_Whitepaper.pdf`, `pqcrypt/latex/PQCrypt_Whitepaper.pdf`. This umbrella document does not duplicate their normative content; it documents the relationship between them, the shared discipline, and the integration patterns that span all three.

Contents

TL;DR **3**

1 The Three Projects **3**
1.1 PQID: post-quantum-era identifier . . . 4
1.2 PQHash: SHAKE-256 content hash . . . 4
1.3 PQCrypt: ML-KEM-1024 encryption envelope . . . 4
1.4 Common infrastructure . . . 5

2 The Shared Variant-Byte Namespace **5**
2.1 Allocation as of v1.x . . . 6
2.2 The cross-rejection contract . . . 6
2.3 Why one namespace and not three . . . 6
2.4 The 9-byte shared header . . . 7
2.5 Encoding overlap . . . 7

3 Canonical Cross-Project Patterns **7**
3.1 Pattern 1: PQID-as-AD (record-level confidentiality binding) . . . 7
3.2 Pattern 2: PQHash-as-AD (content-level confidentiality binding) . . . 8
3.3 Pattern 3: PQID + PQHash (identifier + content commitment) . . . 8
3.4 Pattern 4: PQHash of a PQCrypt envelope (content-addressed ciphertext) . . . 8
3.5 Pattern 5: PQID-tagged PQHash (provenance-tagged content commitment) . . . 9
3.6 Anti-patterns . . . 9

4 Polyglot Discipline **9**
4.1 The authority order . . . 10
4.2 The eight languages . . . 10
4.3 The cross-language KAT harness . . . 10
4.4 Branch protection . . . 11
4.5 What a spec change looks like . . . 11

5 Conformance and CI **11**
5.1 Repository layout . . . 12
5.2 The hybrid suite 0x06 skip discipline . . . 12
5.3 Latest cross-language state (2026-05-21, Apple M4 Max) . . . 12
5.4 How CI gates the work . . . 12

6 Cross-Project Threat-Model Overview **13**
6.1 What is and isn't "post-quantum" in pqtoolkit . . . 13
6.2 Cross-cutting invariants . . . 15
6.3 Side-channel posture . . . 15
6.4 What pqtoolkit as a whole does not provide . . . 15
6.5 Pre-external-audit posture . . . 16

7 Roadmap and Scope Discipline **16**

7.1 Per-project release state (as of 2026-05-21) 16
7.2 Open backlog (v1.4 / v2) 16
7.3 Scope discipline 17
7.4 What *would* trigger a v2 of the whole toolkit 17

A Quick Reference Card **18**

B Variant-Byte Allocation Table **18**

C PQCrypt Suite-Byte Allocation **18**

D Repository Layout **19**

E Author and Document Provenance **19**

TL;DR

pqtoolkit is a specification-first, polyglot monorepo housing three post-quantum-era data formats:

- PQID (variant bytes `0x01`/`0x02`, 41 B): a sortable, post-quantum-era **identifier**. Pairs a 48-bit ms timestamp with 256 bits of CSPRNG-supplied randomness; the smallest random field that retains 128-bit second-preimage security against Grover's algorithm (Grover, 1996). Two variants: Standard (wall-clock) and HLC (Hybrid Logical Clock) (Kulkarni et al., 2014).
- PQHash (variant byte `0x03`, 73 B): a SHAKE-256-based **content identifier**. Pairs the same 9-byte provenance header with a 64-byte (512-bit) SHAKE-256 (fip, 2015) tail. 2^{256} classical collision, $\sim 2^{170}$ BHT (Brassard et al., 1998) quantum collision. Not a hash function; a wire format built on one.
- PQCrypt (variant byte `0x04`, 16-byte header + variable body): an **encryption envelope**. v1 mandatory suite is ML-KEM-1024 (fip, 2024a) + AES-256-GCM + 32-byte UtC commitment tag + HKDF-SHA3-256. v1.2 added the opt-in hybrid suite `0x06` (X25519 + ML-KEM-1024) for callers who want a classical safety net.

The three projects share one variant-byte namespace, one 9-byte header layout (`TS(6) || VARIANT(1) || CTR(2)`), one cross-language conformance harness at `tools/conformance/run.sh`, one Keccak/SHAKE-256 implementation under `common/`, and one author. Per-project normative content lives in the per-project whitepapers; this document is the cross-project overview.

The integration story

When a record needs a stable identifier, use PQID. When a record's content needs an integrity commitment, use PQHash. When a record's content needs confidentiality, wrap it in PQCrypt with the PQID (or the PQHash) as AEAD associated data. The three formats compose without coupling: a PQCrypt envelope can carry a PQID or a PQHash as AD; a PQHash can be the AD that binds a PQCrypt envelope to specific content; a PQID can identify a record whose serialized form a PQHash commits to and a PQCrypt envelope encrypts. The three projects exist as siblings, not as a layered cake.

Scope of this umbrella document

This whitepaper does *not* restate the normative content of any per-project whitepaper. It documents only the relationships, shared infrastructure, and canonical cross-project patterns. For PQID's 14-section spec, PQHash's 10-section spec, or PQCrypt's 14-section spec, refer to the project-specific PDFs.

1 The Three Projects

Each project under pqtoolkit is independent on the wire: a consumer can adopt one without adopting any other, and a downstream system that cares about only identifiers (PQID) need not pull in SHAKE-256 (PQHash) or ML-KEM-1024 (PQCrypt). The projects share discipline (Section 4), tooling (Section 5), and a few load-bearing infrastructure pieces (Section 2), but their wire formats commit to nothing about each other beyond non-collision of variant bytes.

1.1 PQID: post-quantum-era identifier

Project root: `pqid/`. **Whitepaper:** `pqid/latex/PQID_Whitepaper.pdf` (or repo-root `PQID_Whitepaper.pdf`).

PQID is a 41-byte, lexicographically-sortable identifier. Layout: `TS(6) || VARIANT(1) || COUNTER(2) || RANDOM(32)`, big-endian. The 32-byte random tail is the security claim: 2^{128} second-preimage even against an attacker running Grover's algorithm (Grover, 1996) — the same security target NIST uses for ML-KEM-1024 (fip, 2024a) (which PQCrypt uses; see below). The 16-bit per-millisecond counter and 48-bit timestamp make the format sortable and traceable without coordination.

PQID v1 ships two variants:

- `0x01` (Standard): wall-clock-driven. Not internally locked; one generator per thread, or caller-locks.
- `0x02` (HLC): Hybrid Logical Clock (Kulkarni et al., 2014). Internally locked. 60-second drift bound: a peer HLC more than 60 s ahead of local physical time is a hard error.

Canonical text encodings: 55-character Base64URL (Josefsson, 2006) (no padding) and 66-character Crockford Base32 (Crockford) (case-insensitive on decode, `I/L` → `1` and `O` → `0` confusables accepted). Both round-trip byte-identical with the binary. Crockford ASCII sort preserves generation-time order.

1.2 PQHash: SHAKE-256 content hash

Project root: `pqhash/`. **Whitepaper:** `pqhash/latex/PQHash_Whitepaper.pdf`.

PQHash is a 73-byte content identifier. Layout: `TS(6) || VARIANT(1) || COUNTER(2) || TAIL(64)`, big-endian. The 64-byte tail is `SHAKE-256(content, 64 bytes)` (fip, 2015); the 9-byte header is layout-identical to PQID's header. The security argument is the SHAKE-256 collision-resistance assumption: 2^{256} classical collision, $\sim 2^{170}$ Brassard–Høyer–Tapp (Brassard et al., 1998) quantum collision, 2^{256} Grover second-preimage.

PQHash v1 ships one variant: `0x03`. Canonical text encodings: 98-character Base64URL and 117-character Crockford Base32. PQHash is keyless; it provides no authentication, no confidentiality, and no producer non-repudiation (its threat model is closely tied to "the SHAKE-256 sponge is what NIST says it is"). Two callers hashing the same content produce identifiers that share the 64-byte tail and differ only in the 9-byte header.

1.3 PQCrypt: ML-KEM-1024 encryption envelope

Project root: `pqcrypt/`. **Whitepaper:** `pqcrypt/latex/PQCrypt_Whitepaper.pdf`.

PQCrypt is a 16-byte-header variable-body encryption envelope. Header layout: `TS(6) || VARIANT(1) || SUITE(1) || COUNTER(2) || KEM_CT_LEN(2) || AD_LEN(4)`; body: `KEM_CT || AD || NONCE(12) || AEAD_CT || AEAD_TAG(16) || COMMIT(32)`. The first 9 bytes are layout-compatible with PQID's and PQHash's headers; bytes 9–15 are PQCrypt-specific.

PQCrypt v1.x ships three suites:

- `0x01` (mandatory, pure PQ): ML-KEM-1024 + AES-256-GCM + 32-byte UtC commitment tag (Bellare and Hoang, 2022) + HKDF-SHA3-256.

- `0x02` (optional, pure PQ): ML-KEM-1024 + ChaCha20-Poly1305 + UtC + HKDF-SHA3-256. For software-only callers without AES hardware.
- `0x06` (optional, hybrid; v1.2, PQCK-00021): X25519 (Langley et al., 2016) + ML-KEM-1024 + AES-256-GCM + UtC + HKDF-SHA3-256. Opt-in defense-in-depth; uses an X-Wing–style (Connolly et al., 2025) SHAKE-256 combiner over both shared secrets.

The suites `0x01`/`0x02` carry an unambiguous pure-PQ confidentiality promise: residual risk (a structural break of Module-LWE) is accepted in exchange. Suite `0x06` is the opt-in alternative that adds a classical safety net via X25519. The wire byte at header offset 7 records which promise the envelope makes; a single envelope never makes both.

1.4 Common infrastructure

Project root: `common/`. Not a normative wire format; a shared library directory whose contents are used by more than one project.

- `common/c/keccak.{h,c}` (and `common/java/com/pqtoolkit/common/Keccak.java`): SHAKE-256/Keccak. Used by PQHash (tail computation) and PQCrypt (HKDF salt derivation; hybrid suite combiner). Python and Go use stdlib (`hashlib.shake_256`, `golang.org/x/crypto/sha3`); Rust and TypeScript use audited crates. The `common/` copy is the source of truth for C, C++, and Java.
- `common/c/pqtk_header.h`: shared offset constants for the 9-byte header layout (`TS`, `VARIANT`, `COUNTER`). All three projects parse the same first nine bytes the same way.
- `common/c/random.{h,c}`: OS CSPRNG wrapper. Used by PQID (random tail) and PQCrypt (ML-KEM `Encaps` randomness, X25519 ephemeral keygen).
- `common/c/hkdf_sha3.{h,c}`: HMAC-SHA3-256 and HKDF-SHA3-256. PQCrypt-only as of v1.3, vendored alongside the Keccak source.

The discipline: code lives under `common/` only if more than one project depends on it today. The Keccak source moved to `common/` when PQCrypt became its second consumer (PQHash alone could vendor it inside the project). The HKDF-SHA3 wrapper has not moved out of `common/`-PQCrypt-territory because no other project uses it.

2 The Shared Variant-Byte Namespace

The single most important piece of shared infrastructure in pqtoolkit is not a library — it is a one-byte namespace. The variant byte at offset 6 of every wire format names one of the three projects (or a permanent-reserved sentinel). Cross-project decoders reject every byte they do not own.

2.1 Allocation as of v1.x

Byte	Owner	Meaning	Status
0x00	—	Permanently reserved (never assign)	MUST reject
0x01	PQID	Standard generator	assigned v1.0
0x02	PQID	HLC generator	assigned v1.0
0x03	PQHash	SHAKE-256 / 64-byte tail	assigned v1.0
0x04	PQCrypt	Encryption envelope (suite byte at offset 7)	assigned v1.0
0x05–0xFE	—	Reserved-future	MUST reject
0xFF	—	Permanently reserved (never assign)	MUST reject

The reservation of 0x00 and 0xFF is defensive: zero-filled or all-ones buffers cannot pass validation as any project's wire form. The reserved-future range 0x05–0xFE is available for additional projects under pqtoolkit; each new allocation requires a normative whitepaper update and conformance-vector additions to every existing project (to verify cross-rejection).

2.2 The cross-rejection contract

Every project's decoder MUST reject every variant byte it does not own:

- PQID decoders accept 0x01 and 0x02 only. Bytes 0x03 (PQHash), 0x04 (PQCrypt), and every reserved-future byte produce `UnknownVariant` / `PQID_E_UNKNOWN_VARIANT`.
- PQHash decoders accept 0x03 only. Bytes 0x01, 0x02, 0x04, and reserved-future bytes produce `PQHASH_E_UNKNOWN_VAR`.
- PQCrypt decoders accept 0x04 only. Bytes 0x01, 0x02, 0x03, and reserved-future bytes produce `UnknownVariant` / `PQCRYPT_E_UNKNOWN_VARIANT`.

The contract is gated by `invalid.json` KAT files in every project's `conformance/vectors/` directory. A decoder that accepts a foreign variant byte fails the cross-language harness; the bug is in the decoder, not in the harness.

2.3 Why one namespace and not three

A naive design would let each project define its own variant byte in its own one-byte field. We chose a single shared namespace for three reasons:

1. **Confusion attacks.** If a decoder sees the first nine bytes of a wire form and the four bytes "timestamp + variant + counter" happen to look valid by every project's local rules, the decoder must decide which project to dispatch to. With a single namespace, the variant byte is the dispatcher; there is no ambiguity.
2. **Substitution attacks.** An attacker who knows a system accepts PQIDs as primary keys and PQHashes as content keys could substitute one for the other if the variant bytes were locally-scoped. With cross-rejection enforced by the namespace, the substitution fails at the decoder.
3. **Future allocation discipline.** A single, externally-visible byte allocation table makes it impossible to retroactively claim "oh, our project always meant 0x01 to be *this*" — the table is normative across projects, version-controlled, and tested.

2.4 The 9-byte shared header

The first 9 bytes of every project's wire format are layout-compatible: `TS(6) || VARIANT(1) || COUNTER(2)`, big-endian.

- `TS(6)`: 48-bit Unix millisecond timestamp. Every project uses the same constant `MIN_TIMESTAMP_MS = 1_700_000_000_000` as a sanity-check lower bound.
- `VARIANT(1)`: the cross-project namespace byte.
- `COUNTER(2)`: 16-bit big-endian monotonic counter within a millisecond.

A single C header at `common/c/pqtk_header.h` pins those offsets. Code that just wants to read the timestamp from any pqtoolkit wire form parses the first 6 bytes without knowing or caring which project produced it. Code that wants to dispatch on project reads byte 6.

The 9-byte boundary is where the projects diverge: PQID writes a 32-byte random tail, PQHash writes a 64-byte SHAKE-256 tail, and PQCrypt writes a 7-byte body-header continuation (suite + KEM_CT_LEN + AD_LEN) followed by a variable body.

2.5 Encoding overlap

PQID and PQHash both encode to Base64URL (RFC 4648 §5) and Crockford Base32. The encoder code paths are identical across the two projects in every language (with different alphabet sizes per project). PQCrypt ships binary-only on the wire: envelope sizes vary, the variable-length text-encoder is more involved than the fixed-length ones, and most callers wrap envelopes with standard Base64URL at the transport layer if they need text.

PQID and PQHash both apply the same padding-bit guards on decode (Base64URL: final character low-4-bits zero; Crockford: final character low-bit zero). These guards prevent multiple distinct text strings from decoding to the same binary, which would violate canonical round-trip.

3 Canonical Cross-Project Patterns

The three projects are designed to compose. This section documents the canonical patterns and the reasoning behind each.

3.1 Pattern 1: PQID-as-AD (record-level confidentiality binding)

Use when: a record has a stable PQID identifier and the record's content needs confidentiality and integrity tied to that identifier.

```
record_id  = pqid.new_hlc()                                  # 41 bytes, variant 0x02
envelope   = pqcrypt.encrypt(plaintext, recipient_pub,
                             associated_data=record_id)  # bind PQID as AEAD AD
# ... after delivery ...
plaintext  = pqcrypt.decrypt(envelope, recipient_priv,
                             expected_ad=record_id)      # MUST match
```

What it defends: an attacker who reroutes the envelope under a different PQID (e.g., to make

a leaked envelope appear to belong to a different record) gets AEAD-AD mismatch on decrypt, which produces PQCrypt's generic `DecryptionError`. The envelope is cryptographically bound to the identifier it was issued for.

Cost: zero additional cryptography. The AEAD AD path is free in every AEAD construction. The PQID is 41 bytes; the AEAD AD field has a 64 KiB cap (PQCrypt `AD_LEN` $\leq$ 65,536).

Test coverage: `pqcrypt/conformance/vectors/ad_binding.json` exercises positive (matching AD decrypts) and negative (any AD bit-flip $\rightarrow$ `DECRYPT_FAILED`) cases.

This is the most-used cross-project pattern, named the "canonical pattern" in PQCrypt's normative whitepaper.

3.2 Pattern 2: PQHash-as-AD (content-level confidentiality binding)

Use when: the encrypted payload must be bound to specific content bytes, not just "some content this recipient can decrypt."

```
content_hash = pqhash.hash_only(plaintext)                # 64 bytes
envelope     = pqcrypt.encrypt(plaintext, recipient_pub,
                               associated_data=content_hash)
# ... after delivery ...
plaintext    = pqcrypt.decrypt(envelope, recipient_priv,
                               expected_ad=content_hash)
```

What it defends: an attacker who substitutes the envelope with one carrying different content (but the same recipient key and structurally-valid wire form) fails decryption because the AEAD AD they would supply does not match the embedded content. The pattern composes with UtC commitment (PQCrypt's 32-byte commitment tag, (Bellare and Hoang, 2022)): UtC closes multi-recipient key confusion; PQHash-as-AD closes content substitution. The two bindings are independent.

Variant: instead of passing just the 64-byte SHAKE-256 tail, pass the full 73-byte PQHash as AD. This carries provenance (the PQHash's timestamp and counter) alongside the content commitment. The choice depends on whether the AD-binding consumer needs the provenance metadata.

3.3 Pattern 3: PQID + PQHash (identifier + content commitment)

Use when: a record-keeping system tracks both record identity (which mutates rarely) and content (which mutates often).

```
record_id    = pqid.new_hlc()           # immutable for the record's lifetime
content_hash = pqhash.hash_only(...)    # rolls on every content update
# Persist {record_id, content_hash, body} as the canonical record view.
# Subscribers verify both: record_id pins identity, content_hash pins content.
```

What it defends: content-substitution attacks at the storage layer. An attacker who modifies a record's body without rolling the PQHash produces inconsistent state that storage-level integrity checks detect. Independent of any cryptographic envelope.

3.4 Pattern 4: PQHash of a PQCrypt envelope (content-addressed ciphertext)

Use when: an object-storage system indexes by ciphertext rather than plaintext.

```
envelope     = pqcrypt.encrypt(plaintext, recipient_pub)
storage_key  = pqhash.hash_only(envelope)                # 64-byte SHAKE-256
# Store at key=storage_key; lookup by
    key=pqhash.hash_only(known_envelope_bytes).
```

What it defends: accidental duplication of identical ciphertexts under different storage keys. PQCrypt envelopes are deterministic in their inputs (plaintext, recipient key, AD, sender randomness): two envelopes with the same input randomness are byte-identical, so PQHash-keying coalesces them. With different sender randomness, two envelopes for the same plaintext differ, so deduplication does not collapse them — this is the right behavior for a system that wants per-encryption auditability.

3.5 Pattern 5: PQID-tagged PQHash (provenance-tagged content commitment)

Use when: content has multiple producers and the audit log needs to record "producer X committed to content Y at time T."

```
record_id = pqid.new_hlc()              # who is committing
content_h = pqhash.hash_only(content)   # what they are committing to
audit_entry = sign(record_id || content_h, producer_signing_key)
# Persist audit_entry. Note: PQCrypt is not involved if the content is public;
# wrap with PQCrypt only when confidentiality is also required.
```

Note: pqtoolkit does not (yet) ship a signature primitive. The signing step is performed with an external scheme: ML-DSA (fip, 2024b) for FIPS-aligned PQ signing, Ed25519 (Josefsson and Liusvaara, 2017) for classical, or a hybrid via PASETO (Paragon Initiative Enterprises). A future `pqsign`-style sibling project is outside v1.x scope.

3.6 Anti-patterns

- **Using PQID as the AEAD key.** PQID is a public identifier, not a key. Anyone who has seen the PQID can forge an envelope under it. Use a proper key from a plugin (PQCrypt's `pqcrypt-plugin-file` or an HSM-backed plugin).
- **Using PQHash as a password.** PQHash's tail is content-deterministic and keyless. An attacker who can guess the content recomputes the tail. Use Argon2id (Biryukov et al., 2021) or scrypt for password derivation; PQHash's threat model explicitly excludes secrecy of low-entropy content.
- **Truncating to fewer bytes than the wire format prescribes.** PQID is exactly 41 bytes; PQHash is exactly 73 bytes; PQCrypt's header is exactly 16 bytes. "Optimizing" by dropping fields breaks the wire contract and the cross-language harness.
- **Mixing variant bytes within an artifact.** A 41-byte buffer whose variant byte is `0x03` is not a PQID; it is a malformed input that the PQID decoder rejects (and the PQHash decoder also rejects, because the length is wrong). Don't try to "upgrade" a PQID into a PQHash by editing one byte.

4 Polyglot Discipline

Every pqtoolkit project ships byte-identical reference implementations in eight languages, all consumed by the same cross-language KAT harness. This section documents the discipline that makes that work.

4.1 The authority order

When any two sources disagree, conflicts are resolved in the following order:

1. The project's whitepaper LaTeX (this document for cross-cutting concerns; the per-project document for project-specific normative content).
2. The project's Python reference: `pqid/python/pqid.py`, `pqhash/python/pqhash.py`, `pqcrypt/python/pqcry`
3. The project's canonical C ABI header: `pqid/c/pqid.h`, `pqhash/c/pqhash.h`, `pqcrypt/c/pqcrypt.h`.
4. Every other language port.

A cross-language disagreement is never resolved by changing the spec to match an outlier. The outlier is fixed. "Python is right" is the short version of the rule, with whitepaper-overrides-Python and C-overrides-other-languages as the tie-breakers.

4.2 The eight languages

Every project ships reference implementations in:

Language	Role	Why
Python	Readable canonical reference	The spec is the same shape as the Python; the Python is the test
C	Canonical ABI	Offsets, error codes, function signatures pinned in `pqcrypt.h` / `pq`
C++	C-library wrapper	RAII surface over C; no normative content of its own.
Go	Stdlib-heavy port	Tests that the spec is implementable using only stdlib + audited
Rust	Strongest constant-time toolchain	`#![forbid(unsafe_code)]`; tests the safety story.
TypeScript	Browser + Node	Tests that the spec is implementable in JavaScript runtimes.
Java	JVM	Tests that the spec works against JDK stdlib (SunJCE) where ap
CPython extension	Optimized Python	Mirrors the Python API one-to-one; provides a C-fast option with

For PQCrypt specifically (post-quantum, more moving parts than the other two projects), the alpha language set during v1.x development was Python + C + Go + Rust — the minimum that exercises a readable reference, a canonical ABI, a stdlib-heavy ecosystem, and the strongest constant-time toolchain. C++/TypeScript/Java/CPython ext joined in v1.1.

4.3 The cross-language KAT harness

`tools/conformance/run.sh` is the single entry point. It discovers per-language runners under `tools/conformance/python/`, `tools/conformance/c/`, etc., builds each one if needed, and runs each against every KAT file in `pqid/conformance/vectors/`, `pqhash/conformance/vectors/`, and `pqcrypt/conformance/vectors/`.

```
# Full run: all eight languages times all three projects
bash tools/conformance/run.sh
```

```
# Subset of languages
bash tools/conformance/run.sh python rust go

# Skip languages whose toolchain isn't available locally
PQTK_CONFORMANCE_SKIP="cpython java" bash tools/conformance/run.sh

# PQCrypt v1.x alpha quick loop
PQTK_CONFORMANCE_SKIP="cpython java cpp typescript" bash
    tools/conformance/run.sh
```

4.4 Branch protection

`.github/workflows/conformance.yml` runs the harness on every PR against `main`. Red harness ⇒ no merge. Per-language workflows (`python.yml`, `c.yml`, ...) cover per-OS and per-toolchain variants independently; the cross-language harness is the only gate that catches polyglot drift.

PQCrypt has an additional per-project workflow (`pqcrypt.yml`, added in PQCK-00023) that runs the PQCrypt-only matrix across Ubuntu and macOS for the alpha languages plus the LaTeX whitepaper build.

4.5 What a spec change looks like

A normative change to any project follows this order:

1. Update the project's LaTeX whitepaper (and, if cross-cutting, this umbrella document).
2. Update the Python reference.
3. Update the C reference (header first if offsets/constants change, then `.c`).
4. Port to every other language in lockstep. Each must produce byte-identical output for the same input as Python.
5. Regenerate conformance vectors from the Python reference; commit verbatim.
6. Run `bash tools/conformance/run.sh`; confirm green.
7. Write a release-notes entry under `<project>/docs/release-notes/YYYY-MM-DD-<slug>.md`; append to `<project>/docs/release-notes/CHANGELOG.md`.
8. For security-relevant changes, tag the PR `sec(<TICKET>)` and add a "Security considerations" section to the release-notes entry.

Optimization PRs that only touch one language are fine and don't require porting, as long as observable behavior is identical and KAT vectors still pass byte-identical.

5 Conformance and CI

The cross-language conformance harness is the single quality signal that catches polyglot drift. This section documents how it works at the umbrella level; per-project KAT vector inventories live in the per-project whitepapers.

5.1 Repository layout

Path	Contents
`pqid/conformance/vectors/`	PQID KAT files (decode, encode/decode round-trip, invalid, HLC overflow, min t
`pqhash/conformance/vectors/`	PQHash KAT files (decode, hash_only, encode/decode B64URL, encode/decode
`pqcrypt/conformance/vectors/`	PQCrypt KAT files (decrypt, ad_binding, commitment, kyberslash_regression, i
`tools/conformance/run.sh`	Single cross-language entry point
`tools/conformance/<lang>/`	Per-language runner sources and binaries
`.github/workflows/conformance.yml`	Branch-protection gate on `main`
`.github/workflows/pqcrypt.yml`	PQCrypt-only matrix CI (PQCK-00023)

5.2 The hybrid suite `0x06` skip discipline

PQCrypt's hybrid suite `0x06` (PQCK-00021, v1.2) is a polyglot KAT contract that some language ports have not yet caught up to as of v1.3. The harness filters `hybrid_*.json` vector files per language via the `hybrid_supported()` function in `tools/conformance/run.sh`:

```
hybrid_supported() {
    case "$1" in
        python) return 0 ;;   # PQCK-00021, v1.2
        rust)   return 0 ;;   # PQCK-00028, v1.3
        go)     return 0 ;;   # PQCK-00029, v1.3
        *)      return 1 ;;   # C / C++ / TS / Java / CPython ext: v1.4 backlog
    esac
}
```

Languages without hybrid skip the `hybrid_*.json` files entirely and MUST reject suite `0x06` as `UNKNOWN_SUITE` for any non-hybrid envelope they encounter. When porting hybrid to a new language: (i) implement hybrid in that language, (ii) update the language's conformance runner to accept 96-byte hybrid seeds (the standard runners hard-coded `len(seed) != 64`), (iii) flip the language's `hybrid_supported()` entry, (iv) verify the language's PASS count rises by 9 (one per record across the three hybrid KAT files).

5.3 Latest cross-language state (2026-05-21, Apple M4 Max)

Language	PASS	FAIL	Notes
Python	265	0	PQID + PQHash + PQCrypt v1.x including hybrid `0x06`.
C	256	0	PQCrypt hybrid skipped; PQCK-00030 backlog.
Go	265	0	PQCrypt hybrid via stdlib `crypto/ecdh` (PQCK-00029).
Rust	265	0	PQCrypt hybrid via `x25519-dalek` 2 (PQCK-00028).

C++, TypeScript, Java, and CPython ext consume PQCrypt vectors via the Python runner fallback (`pqcrypt_runner_cmd` empty case in `tools/conformance/run.sh`) and skip `hybrid_*.json`. Their per-language test suites still cover PQID and PQHash directly.

5.4 How CI gates the work

Every PR to `main` runs the harness across all available language toolchains in the GitHub Actions matrix. The harness is non-flaky by design: KAT vectors are deterministic, the per-language runners

do not call out to the network, and the toolchains are pinned. A red harness means one of two things:

- A language port produced a different byte sequence for the same input. The port is wrong; fix the port.
- The KAT vector itself changed. Re-running `<project>/conformance/scripts/generate.py` produces non-identical JSON. The Python reference's behavior changed; this is a spec change, requires a release-notes entry, and all other language ports must be updated to match before the PR merges.

The harness is what makes the spec-first methodology load-bearing. Without it, "Python is right" is just a slogan; with it, a language port that disagrees with Python is detectable in seconds.

6 Cross-Project Threat-Model Overview

Each project has its own normative threat model in its own whitepaper. This section summarizes the differences and the shared ground.

6.1 What is and isn't "post-quantum" in pqtoolkit

The "post-quantum" in pqtoolkit means different things in different projects, and the differences matter.

Project	What "PQ" refers to	Binding security argument
PQID	*Post-quantum-era hygiene.* The 256-bit random tail retains 128-bit second-preimage against Grover's algorithm (Grover, 1996). PQID is not a cryptographic primitive; it is an identifier whose random field is sized for the era in which quantum	"The OS CSPRNG produces unguessable bits, and 256 of them is enough."

The first two are not making cryptographic claims — they are public artifacts. An attacker sees every byte of every PQID and PQHash; the security of PQID reduces to "the random tail is unguessable" and the security of PQHash reduces to "the SHAKE-256 hash is collision-resistant." Both projects defer producer authentication, integrity of the wire form, and confidentiality of their contents to an outer envelope.

PQCrypt is the only project that makes a cryptographic confidentiality claim. The claim is structured carefully: suites `0x01`/`0x02` make a *pure post-quantum* promise (no classical fallback; if ML-KEM-1024 is structurally broken, every envelope under these suites loses confidentiality). Suite `0x06` makes a *hybrid* promise (an attacker must break both ML-KEM-1024 *and* X25519 to recover the session key). The wire byte at offset 7 records which promise applies; a single envelope never makes both.

6.2 Cross-cutting invariants

These apply to all three projects:

- **OS CSPRNG only.** Where randomness is needed (PQID's 32-byte tail; PQCrypt's ML-KEM `Encaps` randomness and X25519 ephemeral keygen for hybrid), it comes from the OS CSPRNG: `getrandom(2)` / `getentropy(3)` / `BCryptGenRandom` / `os.urandom` / `SecureRandom` / `crypto.getRandomVal` / `/dev/urandom`. No userspace PRNG, no fallback. PQHash has no random field and is exempt.
- **Variant-byte cross-rejection.** Each project's decoders reject every variant byte they do not own (Section 2). The contract is gated by KAT vectors.
- **No unsafe Rust, no cgo, no new FFI.** Implementations are pure within their language. CPython extensions are direct C against the CPython API, not FFI bridges.
- **Supply-chain hygiene.** No new third-party crypto dependencies. SHAKE-256/Keccak comes from stdlib (Python, Go) or vendored `common/` (C, C++, Java). PQCrypt's ML-KEM-1024 and X25519 sources are pinned per language; see PQCrypt's whitepaper §5.

6.3 Side-channel posture

- PQID and PQHash are *public artifacts* with no secret-dependent control flow. Side-channel attacks on the generator recover, at most, content the attacker may already have submitted (PQHash) or the rate of identifier generation (PQID).
- PQCrypt mitigates *software-timing* side channels by vendoring constant-time ML-KEM-1024 and X25519 implementations per language and by maintaining a constant-time decrypt path (single `DecryptionError` class, synthetic-key AEAD verify on structural failures). PQCrypt explicitly does *not* mitigate power, electromagnetic, frequency-scaling (Hertzbleed), or prefetcher (GoFetch) side channels; the normative side-channel disclaimer is at `pqcrypt/docs/side-channel-disclaime`

6.4 What **pqtoolkit** as a whole does not provide

- **No signature primitive.** Producer authentication is out of v1.x scope. Use ML-DSA (fip, 2024b), Ed25519 (Josefsson and Liusvaara, 2017), or PASETO (Paragon Initiative Enterprises). A future `pqsign`-style sibling project is outside v1.x scope.

- **No transport layer.** pqtoolkit ships wire formats, not protocols. Use TLS 1.3, QUIC, or Noise underneath.
- **No FIPS 140-3 module validation.** The algorithms used in PQCrypt are FIPS-approved (FIPS 203 ML-KEM, FIPS 197 AES, FIPS 202 SHA-3 family); the implementation is not a validated module. Callers who need module validation should source ML-KEM from a validated module (e.g., a future BoringCrypto release) and integrate at the API boundary.
- **No key rotation primitive.** Each PQCrypt envelope is self-contained; rotation is a higher-level concern that the plugin protocol and the outer application own.

6.5 Pre-external-audit posture

PQCrypt v1.x ships with an internal pre-external-audit threat-model walk-through at `pqcrypt/docs/security-audi`
It enumerates known attacks vs mitigations, per-language posture, and six open items for an external auditor (CI timing histogram, FIPS 140-3 path, synthetic-key timing-pad invariant testing, hybrid C port, plugin protocol fuzz harness, memory-zeroization audit).

PQID and PQHash are simpler in shape and do not have an equivalent internal audit document; their threat models live entirely in their respective whitepapers. A future external audit would cover all three projects, with the PQCrypt portion being the largest by surface area.

7 Roadmap and Scope Discipline

This section documents what is in scope for v1.x, what is scheduled for v1.4/v2, and what is explicitly out of scope.

7.1 Per-project release state (as of 2026-05-21)

Project	Version	Date	Major milestones
PQID	v1.1	2026-05-21	Eight-language byte-identical implementations; HLC variant; cross-project variant-byte rejection contract pinned.
PQHash	v1.0	2026-05-21	Eight-language byte-identical implementations; SHAKE-256 tail; cross-rejection of cross-project bytes.
PQCrypt	v1.3 partial	2026-05-21	Eight-language alpha for suites `0x01`/`0x02`; hybrid `0x06` in Python, Rust, Go; plugin protocol + `.pqck-key-v1.1` keyfile; CLI binary.

7.2 Open backlog (v1.4 / v2)

PQCrypt v1.4 (priority order):

1. PQCK-00030: Hybrid suite `0x06` port to C (OpenSSL 3.5+ `EVP_PKEY_X25519`). Closes the largest single gap in the cross-language hybrid contract.
2. PQCK-00031: Hybrid `0x06` in TypeScript (`@noble/curves/ed25519` X25519).
3. PQCK-00032: Hybrid `0x06` in Java (SunJCE `XDH` keypair, JEP 339).
4. PQCK-00033: Hybrid `0x06` in CPython extension (mirror Python).

5. PQCK-00034: Rust AES-NI/ARMv8 intrinsics for 1 MiB AEAD throughput. Currently regressed because `aes-gcm 0.10` does not enable ARMv8 intrinsics on stable Rust without a nightly cfg.
6. PQCK-00035: CI timing-histogram regression test. Currently the histogram check is manual; automating it in CI closes open item #1 from the v1.2 security audit doc.

PQID v1.x: no open backlog. The format is stable; future work is operational (benchmarks, additional reference implementations, documentation polish).

PQHash v1.x: CPython extension is the largest single gap; mirrors the Python API.

Cross-project: sibling project `pqsign` (variant byte `0x05`? or further into the reserved-future range) is an explicit *out-of-v1.x* candidate. The minimal scope would be ML-DSA (fip, 2024b) signatures of a payload-with-context, using a PQID or PQHash as the canonical payload. Not committed; documented as a possibility so the reserved-future range is interpretable.

7.3 Scope discipline

- **No new variant bytes without a whitepaper update.** `0x00` and `0xFF` are permanently reserved; `0x05`–`0xFE` are reserved-future. Any new allocation requires (a) a normative whitepaper update in the owning project, (b) conformance-vector additions to every existing project verifying cross-rejection, and (c) sign-off as a security-relevant change.
- **No third-party crypto dependencies without justification.** Each project's whitepaper pins per-language sources for its primitives. New dependencies require explicit justification in the PR description.
- **No unsafe Rust, no cgo, no FFI shortcuts.** Implementations stay pure within their language. The CPython extensions are C-against-CPython-API, not FFI bridges.
- **No format-version mutations of an existing variant.** A variant byte *is* a version. A change to the layout, the security argument, or the algorithm choices under an existing byte is a spec break. New algorithms get new bytes (PQCrypt's hybrid suite is at `0x06` not `0x01` for exactly this reason).

7.4 What *would* trigger a v2 of the whole toolkit

- **A structural break of Module-LWE.** Would force PQCrypt to allocate a new core suite (likely under a new variant byte, e.g., a hypothetical PQCrypt v2 at variant `0x06`–no wait, that's already taken; the actual choice would be a new reserved-future allocation). PQID and PQHash are unaffected by ML-KEM breaks because they make no PQ-cryptography claim.
- **A serious break of SHAKE-256.** Would force PQHash to a new variant byte with a different hash function (KMAC256? a stream cipher in PRF mode? a different sponge?). PQCrypt's KDF schedule would also need revision because HKDF-SHA3-256 depends on SHAKE-256. PQID is unaffected.
- **A serious break of AES-256-GCM.** Would force PQCrypt to a new suite under a new byte using a different AEAD. PQID and PQHash are unaffected.

- **A change to the variant-byte allocation discipline.** Would force every project to a new wire-format version, indicated by a new variant byte. This is the worst-case scenario and is exactly what the immutable-variant-byte rule is designed to prevent.

None of these is anticipated in the v1.x lifetime. The roadmap is conservative on purpose: pqtoolkit is intended to outlive any individual deployment, which means most of its versions look like "a new entry was added to the variant table; everything else is unchanged."

A Quick Reference Card

Property	PQID	PQHash	PQCrypt
Variant byte	`0x01` / `0x02`	`0x03`	`0x04` (+ suite byte at offset 7)
Wire size	41 B fixed	73 B fixed	16-byte header + variable body
Tail / body	32 B CSPRNG random	64 B SHAKE-256	`KEM_CT || AD || nonce || CT || tag || COMMIT`
Primitive	OS CSPRNG	SHAKE-256 (FIPS 202)	ML-KEM-1024 (+ X25519 for `0x06`) + AES-256-GCM + UtC
Security claim	128-bit Grover second-preimage	2^{256} classical / $\sim 2^{170}$ BHT collision	ML-KEM-1024 IND-CCA2 + AEAD + UtC commitment
Public artifact?	Yes (no secrets)	Yes (no secrets)	No (carries confidential plaintext)
Text encodings	55-char Base64URL, 66-char Crockford	98-char Base64URL, 117-char Crockford	Binary only (v1)
Round-trip determinism	N/A (random)	Yes (content-deterministic)	Yes given fixed RNG seed (KAT vectors)

B Variant-Byte Allocation Table

Byte	Owner	Project / variant	First spec version	Status
`0x00`	—	Permanently reserved (defensive)	—	MUST reject
`0x01`	PQID	Standard generator	v1.0	assigned
`0x02`	PQID	HLC generator	v1.0	assigned
`0x03`	PQHash	SHAKE-256 / 64-byte tail	v1.0	assigned
`0x04`	PQCrypt	Encryption envelope (suite byte at offset 7)	v1.0	assigned
`0x05`–`0xFE`	—	Reserved-future	—	MUST reject
`0xFF`	—	Permanently reserved (defensive)	—	MUST reject

C PQCrypt Suite-Byte Allocation

(Header offset 7 within a PQCrypt envelope; only valid when `VARIANT == 0x04`.)

Suite	Algorithms	Status
0x01	ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1.0 mar
0x02	ML-KEM-1024 + ChaCha20-Poly1305 + UtC commit + HKDF-SHA3-256	v1.0 opti
0x06	X25519 + ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1.2 opti
0x00, 0x03–0x05, 0x07–0xFF	Reserved-future	MUST r

D Repository Layout

```
pqtoolkit/
|-- CLAUDE.md                   # umbrella discipline file
|-- README.md                   # repo entry point
|-- latex/                      # this umbrella whitepaper
|   `-- main.tex, sec*.tex, references.bib, Makefile
|
|-- pqid/                       # Project 1: identifier
|   |-- CLAUDE.md, README.md
|   |-- latex/                  # PQID whitepaper sources
|   |-- python/, c/, cpp/, go/, rust/, typescript/, java/, cpython/
|   |-- conformance/vectors/    # KAT files
|   |-- benchmarks/             # per-language bench data
|   `-- docs/                   # release notes, docs site sources
|
|-- pqhash/                     # Project 2: content hash
|   |-- (mirror PQID layout; latex/ added in PQHash v1.0)
|   `-- ...
|
|-- pqcrypt/                    # Project 3: encryption envelope
|   |-- (mirror PQID layout)
|   |-- plugin/                 # plugin protocol + reference plugin
|   `-- docs/                   # incl. side-channel-disclaimer.md,
    security-audit-v1.2.md
|
|-- common/                     # shared library code
|   |-- c/                      # keccak, pqtk_header, random, hkdf_sha3
|   `-- java/                   # Keccak.java
|
`-- tools/
    `-- conformance/            # cross-language KAT harness + per-language
        runners
```

E Author and Document Provenance

Author: Bahram Samani, pqtoolkit monorepo maintainer.

This umbrella whitepaper is built from LaTeX sources under `latex/` (`main.tex` + `secNN_*.tex`); the build chain is `pdflatex` → `bibtex` → `pdflatex` × 2 per the Makefile. The artifact at the repo root `PQToolkit_Whitepaper.pdf` is regenerated and committed alongside any normative change to the umbrella.

Per-project documents. This umbrella does not restate normative content from the per-project

whitepapers. Refer to:

- PQID: `pqid/latex/PQID_Whitepaper.pdf` (repo root: `PQID_Whitepaper.pdf`). 46-page document covering wire format, security analysis, distributed-systems model, comparisons with UUIDv4/v7/ULID/KSUID/Snowflake, eight-language reference implementations.
- PQHash: `pqhash/latex/PQHash_Whitepaper.pdf` (repo root: `PQHash_Whitepaper.pdf`). 21-page document covering wire format, SHAKE-256 binding, encodings, security argument, conformance, comparison with adjacent hash formats.
- PQCrypt: `pqcrypt/latex/PQCrypt_Whitepaper.pdf` (repo root: `PQCrypt_Whitepaper.pdf`). 26-page document covering wire format, ML-KEM-1024 binding, hybrid suite `0x06`, AEAD + UtC commitment, KDF, AD-binding patterns, failure modes, side channels, plugin protocol, conformance, comparisons.

Authority order (when sources disagree): the per-project whitepaper for project-specific content; this umbrella whitepaper for cross-project content; then the per-project Python reference; then the per-project canonical C ABI; then every other language port. A cross-language disagreement is never resolved by changing the spec to match an outlier.

Conformance gate: `tools/conformance/run.sh` is the branch-protection check on `main`. Latest cross-language run (2026-05-21, alpha quick loop): 1051 PASS, 0 FAIL across all three projects on the four alpha languages.

Acknowledgments. Bellare and Hoang for the UtC commitment construction. The OpenSSL, RustCrypto, Go core team, `kyber-py`, SunJCE, and `@noble` maintainers for the cryptographic primitive sources PQCrypt vendors. The Cryspen team for `mlkem-native` (retained as embedded-target fallback). The X-Wing draft authors (Connolly et al., 2025) for the combiner design idiom PQCrypt's hybrid suite borrows. The KyberSlash, PQShield, and GoFetch disclosure teams, whose findings shape PQCrypt's side-channel disclaimer. The SHAKE-256 / SHA-3 standardization team (Bertoni, Daemen, Peeters, Van Assche) whose primitive is the load-bearing element of every pqtoolkit project.

References

SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, National Institute of Standards and Technology, Aug. 2015. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS PUB 203, National Institute of Standards and Technology, Aug. 2024a. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf.

Module-lattice-based digital signature standard. Technical Report FIPS PUB 204, National Institute of Standards and Technology, Aug. 2024b. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf.

M. Bellare and V. T. Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, 2022. URL https://eprint.iacr.org/2022/1260.

A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson. The memory-hard Argon2 password hash and proof-of-work function. RFC 9106, IETF, 2021. URL https://www.rfc-editor.org/rfc/rfc9106.

G. Brassard, P. Høyer, and A. Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN '98*, 1998. URL https://arxiv.org/abs/quant-ph/9705002.

D. Connolly, P. Schwabe, and B. Westerbaan. X-Wing: General-purpose hybrid post-quantum KEM. draft-connolly-cfrg-xwing-kem, IETF CFRG, 2025. URL https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/.

D. Crockford. Base32. URL https://www.crockford.com/base32.html.

L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996. URL https://arxiv.org/abs/quant-ph/9605043.

S. Josefsson. The Base16, Base32, and Base64 data encodings. RFC 4648, IETF, 2006. URL https://www.rfc-editor.org/rfc/rfc4648.

S. Josefsson and I. Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017. URL https://www.rfc-editor.org/rfc/rfc8032.

S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone. Logical physical clocks. In *Principles of Distributed Systems (OPODIS)*, 2014. URL https://cse.buffalo.edu/tech-reports/2014-04.pdf.

A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, IETF, 2016. URL https://www.rfc-editor.org/rfc/rfc7748.

Paragon Initiative Enterprises. PASETO: Platform-agnostic security tokens, v4. URL https://github.com/paseto-standard/paseto-spec.